10

Agent de Review: quem constrói não valida

Um agent que implementa e revisa o próprio trabalho sempre chega à mesma conclusão: está ótimo. Ele carrega o viés de ter tomado as decisões — revisar a si mesmo é procurar defeito no próprio raciocínio. O agent de review fecha o ciclo de qualidade justamente por ser *outro*: contexto limpo, sem memória das escolhas, lendo o código como um engenheiro que entra no projeto pela primeira vez.

A IDEIA CENTRAL DESTE MATERIAL

O reviewer não corrige nada. Ele identifica, classifica e reporta. Correção é sessão separada — misturar review com correção produz revisões superficiais, onde o agent resolve o fácil e ignora o estrutural.

1 As três garantias do reviewer

- **Somente leitura** — tools restritas a Read/Grep/Glob, nunca escrita ou execução.
- **Revisa contra documentos** — SPEC.md (o que devia ser construído), PLAN.md (critérios de cada task), CLAUDE.md (convenções). Sem eles, vira só análise de estilo.
- **Hook de bloqueio** — mesmo que o modelo "decida" editar, o hook impede (material 11).

2 Estrutura do agent — Claude Code

```
# .claude/agents/code-reviewer.md
---
name: code-reviewer
description: Revisa código dos agents de implementação contra o
             SPEC.md e o PLAN.md. Use após qualquer implementação
             para validar aderência antes de avançar.
tools: Read, Grep, Glob
model: sonnet
hooks:
  PreToolUse:
    - matcher: "Write|Edit|Bash"
      hooks:
        - type: command
          command: "echo 'code-reviewer é somente leitura' >&2 && exit 2"
```

```
---

Você é um engenheiro sênior especializado em code review.
Sua única responsabilidade é revisar — nunca modificar.

Ao ser invocado:
1. Leia o SPEC.md — o que deveria ter sido construído
2. Leia o PLAN.md — os critérios de cada task
3. Leia o CLAUDE.md/AGENTS.md — as convenções do projeto
4. Analise o código contra esses três documentos

Para cada problema, classifique:
- BLOQUEANTE: impede o funcionamento do sistema
- IMPORTANTE: corrigir antes da entrega
- SUGESTÃO: melhoria para o próximo ciclo

Nunca modifique arquivos. Nunca execute comandos.
Apenas leia, analise e reporte.
```

No **OpenCode**, o equivalente vai em `.opencode/agents/code-reviewer.md` com `mode: subagent`, `temperature: 0.1` e as tools de escrita explicitamente desabilitadas (`write: false, edit: false, bash: false`) — mesmo system prompt.

3 Os três níveis de veredicto

NÍVEL	CRITÉRIO	EXEMPLOS
BLOQUEANTE	Impede o funcionamento	Endpoint retornando 500, teste que passa sem testar nada, função nunca chamada, import inexistente, variável de ambiente ausente.
IMPORTANTE	Funciona, mas compromete qualidade/manutenção	Edge case não tratado, convenção violada, código duplicado que vai gerar inconsistência, erro sem tratamento.
SUGESTÃO	Melhoria desejável, não urgente	Função longa a extrair, comentário confuso, teste que cobre só o happy path.

4 O prompt de review

▸ PROMPT DE REVIEW COMPLETO

```
Leia o SPEC.md, o CLAUDE.md (ou AGENTS.md) e o PLAN.md na raiz.

Faça um review completo do código gerado nestes aspectos:

1. ADERÊNCIA AO SPEC
   O implementado corresponde ao especificado?
   Alguma funcionalidade essencial falta?

2. ADERÊNCIA AO CLAUDE.MD/AGENTS.MD
   Convenções de nomenclatura seguidas?
   Regras de ouro respeitadas?

3. COBERTURA DE TESTES
   Todos os testes críticos do PLAN.md existem?
   Testam comportamento ou passam por acidente?

4. QUALIDADE DO CÓDIGO
   Código duplicado? Edge cases não tratados?

5. VEREDICTO
   Classifique cada problema:
   BLOQUEANTE / IMPORTANTE / SUGESTÃO

Não corrija nada — apenas liste os problemas.
```

5 Review automático: duas arquiteturas

Opção 1 — Hook no agent de implementação

O implementador dispara o reviewer ao encerrar, via Stop hook:

```
# frontmatter do backend-agent
hooks:
  Stop:
    - hooks:
        - type: command
          command: ".claude/hooks/backend/trigger-review.sh"

# .claude/hooks/backend/trigger-review.sh
#!/bin/bash
INPUT=$(cat)
[ "$(echo "$INPUT" | jq -r '.stop_hook_active')" = "true" ] && exit 0
claude --agent code-reviewer \
  "Faça o review do que foi implementado agora"
exit 0
```

Limitação: o reviewer roda dentro da mesma sessão — o contexto não fica completamente isolado.

Opção 2 — Orquestrador Builder-Validator (o padrão mais limpo)

▸ PROMPT DO ORQUESTRADOR

```
Leia o PLAN.md e execute a Fase 2 completa
seguindo este fluxo obrigatório para cada task:

1. Despache @backend-agent para implementar a task
2. Aguarde o retorno confirmando conclusão
3. Despache @code-reviewer para revisar — em contexto isolado
4. Aguarde o veredicto
5. Item BLOQUEANTE? Corrija antes da próxima task
6. Sem bloqueantes? Avance

Não pule o step de review em nenhuma task.
```

Aqui o reviewer **nunca vê a conversa de implementação** — só o output final. É o contexto mais limpo possível, e a materialização do padrão Builder-Validator do material 09.

No OpenCode, o disparo automático equivalente é um plugin TypeScript ouvindo `session.idle`: quando um agent de implementação fica ocioso, o plugin invoca `@code-reviewer` com o SPEC.md como referência.

6 Erros mais comuns

ERRO	POR QUE DESTRÓI O REVIEW
Reviewer com ferramentas de escrita	O maior de todos. Reviewer que pode editar, edita — e aí não é review, é refactoring sem crítica. Trave com tools restritas E hook.
Review sem o SPEC no contexto	Sem saber o que deveria existir, ele avalia só qualidade de código — não aderência a requisitos.

Misturar review e correção	Achou bloqueante, corrigiu na hora? A qualidade do review das issues seguintes despenca. Registre, encerre, corrija em sessão nova.
Revisar só no fim do sprint	Problema da Fase 1 se propaga por todas as fases seguintes. Mínimo: review ao final de cada fase; ideal: após cada task (Builder-Validator).
Ignorar as sugestões	Sugestão de hoje é dívida técnica de amanhã. Registre num backlog — não para resolver agora, para não perder.

▸ **COLA PARA FIXAR**

- Quem implementa **não revisa**: o reviewer nasce com contexto limpo e sem o viés das decisões.
- Reviewer **identifica, classifica e reporta** — nunca corrige. Correção é outra sessão.
- Três travas: tools somente leitura + hook de bloqueio + revisão contra **SPEC/PLAN/CLAUDE.md**.
- Veredicto em três níveis: **BLOQUEANTE** (para tudo), **IMPORTANTE** (antes da entrega), **SUGESTÃO** (backlog).
- Arquitetura mais limpa: orquestrador **Builder-Validator** — o reviewer só vê o output final.
- Frequência mínima: **a cada fase**. Revisar só no fim é deixar o erro se multiplicar.